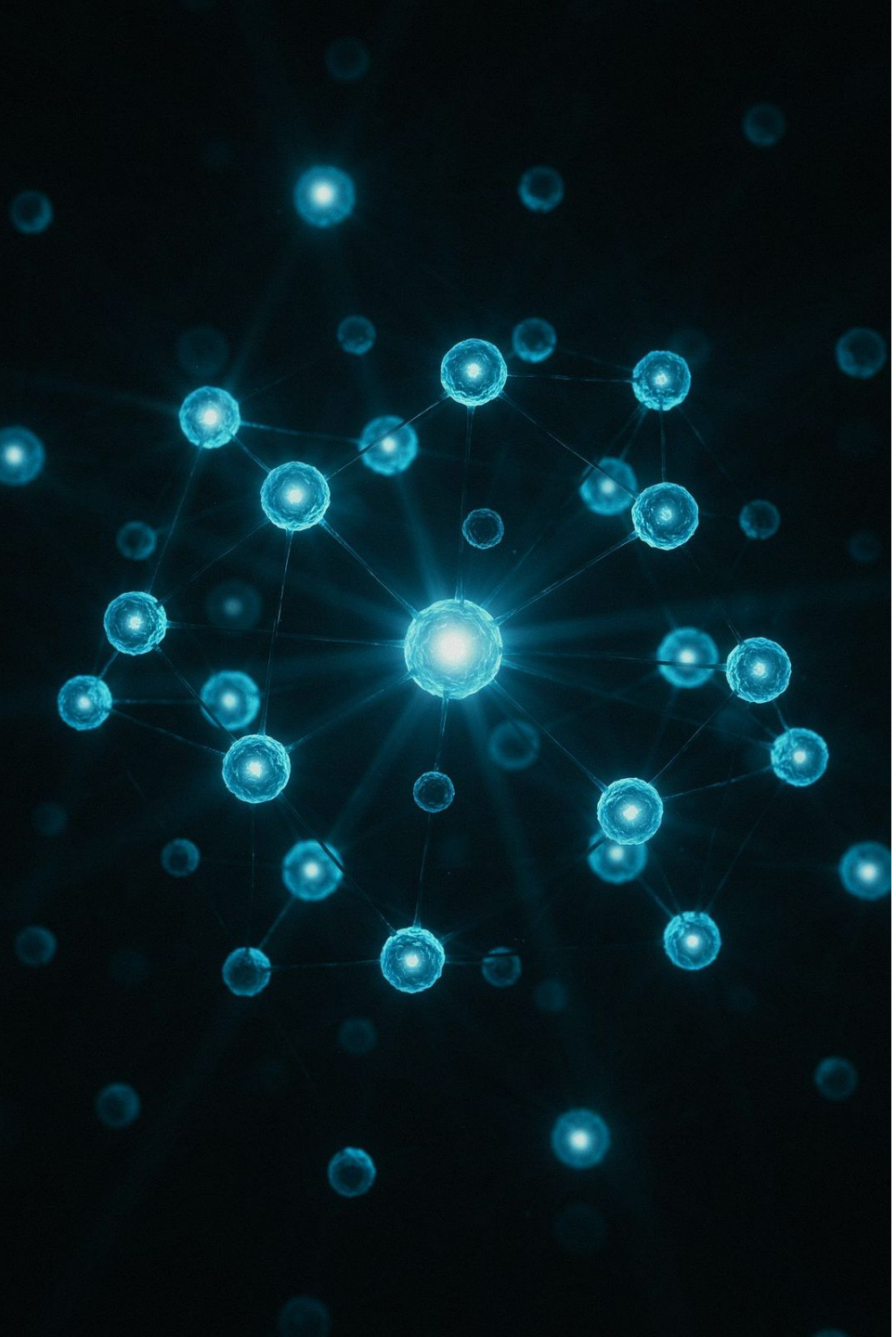


Entrenamiento y regularización

Sprint 10 · Redes neuronales · S10.3

Una red con muchísimos parámetros para pocos datos se aprende los ejemplos de memoria. Aprendemos a verlo y a frenarlo.

Diagnosticar el sobreajuste

Cuaderno S10_03: solo 800 reservas de entrenamiento y una red grande: memorizar es facilísimo.

- La pérdida de entrenamiento sigue bajando.
- La de validación baja, toca un mínimo y vuelve a subir.

```
epoca_optima = int(np.argmin(h_base.history["val_loss"])) + 1   # Ejercicio 1
brecha_final = h_base.history["val_loss"][-1] - h_base.history["loss"][-1]
```

Si epoca_optima es muy inferior al nº de épocas, has entrenado de más.

Las herramientas de regularización

Técnica	Idea en una frase	En Keras
L2 (weight decay)	Castiga los pesos grandes	kernel_regularizer=keras.regularizers.l2(0.01)
Dropout	Apaga al azar un % de neuronas en cada paso	keras.layers.Dropout(0.4)
Parada temprana	Detiene cuando la validación deja de mejorar	EarlyStopping(patience=8, restore_best_weights=True)
Reducir la tasa	Pasos más pequeños al estancarse	ReduceLROnPlateau(factor=0.5, patience=4)

```
m_drop, h_drop = entrenar(A_peq, y_peq, epocas=60, dropout=0.4)   # Ejercicio 2
```

Callbacks, tasa de aprendizaje y lote

```
mis_callbacks = [EarlyStopping(monitor="val_loss", patience=8, restore_best_weights=True),
                 ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=4)]
m_es, h_es = entrenar(A_peq, y_peq, epocas=100, callbacks=mis_callbacks, l2=0.005, dropout=0.3)   # Ejercicio 3
```

- Tasa de aprendizaje muy alta: la pérdida salta y no converge. Muy baja: tarda una eternidad.
- Lote pequeño: más actualizaciones, más ruido. Lote grande: más estable, menos actualizaciones.

Guardar y cargar el modelo

```
m_es.save(ruta_modelo)   # formato .keras (Ejercicio 4)
m_cargado = keras.models.load_model(ruta_modelo)
prob_cargado = m_cargado.predict(A_val)   # idéntico al original
```

- Un modelo solo sirve si se puede reutilizar sin reentrenar.
- En producción hay que aplicar exactamente el mismo preprocesado (pre): lo ideal es guardarlo junto al modelo.

El reto: ¿vale la pena la red?

1. Misma partición

64 % entrenamiento, 16 % validación (decisiones y curvas), 20 % prueba (una sola vez).

2. Referencia clásica

El boosting del cuaderno o tu campeón del Sprint 8, midiendo también el tiempo.

3. MLP con buenas prácticas

Pequeño, con L2, dropout y parada temprana, entrenado con 3 semillas.

- evaluar(y_real, prob, umbral=0.5): AUC, AP, F1 y precisión (Ejercicio 1 de S10_04).
- diagnostico(historial, margen=0.03): devuelve «sobreajuste» o «ok» (Ejercicio 3).

Veredicto y entrega

- Prueba con la media de las 3 semillas; compara con un intervalo de confianza de la diferencia.
- hay_diferencia = inf > 0 or sup < 0 y mejor_modelo ∈ {"MLP", "Boosting", "Empate"} (Ejercicio 4).
- Informe de una página: pregunta y respuesta en una frase, tabla con incertidumbre y coste (tiempo, complejidad), recomendación.

> Las partes marcadas «en casa» (tasa de aprendizaje, guardar el modelo y mini-retos) no son necesarias para el reto.

Ahora, a practicar

Lo que viene a continuación está en tu repositorio.

1 CUADERNO · S10_03_regularizacion (sprints/S10/notebooks/)

Sobreajuste, L2, dropout, parada temprana y callbacks — 55 min.

2 RETO · Esqueleto S10_04 (sprints/S10/notebooks/S10_04_reto_vale_la_pena)

Elegir el problema y reproducir la referencia (paso 1) → primera red (2) → diagnosticar el sobreajuste (3) → comparación justa (4) → informe y presentación (5) — 90 min.